

Figure 1: A database hosting architecture

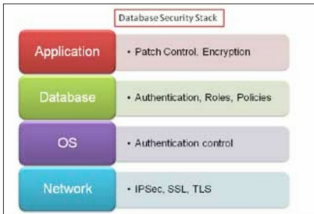


Figure 2: The database security stack

Database hacking

A database contains the mission critical data of a company, which makes it an obvious target for hackers. Please refer to Figure 2, which shows the module stack of the database security model. We will first look at a few ways by which a hacker tries to gain control of a database, followed by a few protection mechanisms that we can deploy to address these problems. Usually, a database server is not hosted in a less secure demilitarised zone; hence, the hacker has to go past the Web layer to access the database.

The database is a widely and continuously accessible component, which makes it more vulnerable and susceptible to attacks. Database security requirements touch all networking layers, and hence, need careful design.

Password guessing: Once the hackers gain control of the Web server, they look for the application configuration files that typically contain the database server name, user ID and password to access it. If the file is encrypted or if the information is hard-coded in the code for security reasons, hackers are left with the option of guessing the password. Often, the database system administrator's account name remains unchanged, which solves half of the hacker's problem! In most cases, the username and password are the same; if not, hackers try guess work or brute force methods to gain access to the database. Depending upon the access level granted to the user, hackers gain control and are capable of doing further damage.

Packet sniffing: Modern hackers gain control of the network in which the database server is hosted, and deploy

a sniffer to capture packets flowing to and from the database server. These packets are then analysed to decipher username and password combinations, with the ultimate aim of getting those of the root database administrator.

Query string manipulation: When a Web server gets a request from a browser, it receives it in the form of a Web URL containing a query string parameter, which directs the Web application to the database to fetch the requested data. If the application's developer is not security aware, and does not follow the best coding practices, this simple method of fetching data can open up loopholes. For example, if a list of doctors is being asked for by an insurance website, the code must ensure that a blank value in the zip code field is not allowed. If it is, the hackers can run a query and fetch the entire data that they normally may not have access to.

Privilege elevation: As mentioned earlier, database servers use roles and rules to control the user's access to a particular database, table or resource. Attackers usually inject scripts that try to find out the most generous level of privileges and upon finding one, use it to create further damage. A few FOSS database servers such as MangoDB were found to be victims of these bugs, which were eventually fixed.

Vulnerability exploitation: Like operating systems, database servers, too, have bugs resulting in serious vulnerabilities. Attackers can explore and exploit those either through the Web layer, or by injecting a Trojan in the system. Hackers are always looking for unpatched database systems for this purpose.

SQL injection: This attack is not a pure database attack, but in fact a fairly well-known combination of database and Web vulnerability. The attackers use Web pages to plant the attack. It is done by including parts of SQL query statements in the Web form, which is usually to be filled up by the Web user with relevant information. If the Web page is not capable of handling this situation in a secure manner, it ends up sending this bogus request to the database. The attackers can simply run a select query to dump the entire database on their machine by this method, which makes it a favourite of hackers and, hence, dangerous from a firm's security standpoint.

Database DoS: As a typical denial of service attack, the attackers plant a code either on the Web server or the database server itself, which sends requests to the database in order to overwhelm it. For example, the queries can be either at the application layer to dump database contents in a loop, or can be a SYN packet storm at the TCP layer. The end result is that the database gives up at one point and stops responding to the queries, leading to a complete website or application failure.

Protecting FOSS systems

Since database servers are an important component of the whole system, multiple measures need to be deployed to ensure security. The following few methods help achieve that.

Patching is the first level of defence for databases. A robust patching system must be deployed to secure all the